Seasonal Predator-Prey Simulation

Final Report

Munir Gargour

CS 4632: Modeling and Simulation

Department of Computer Science

Kennesaw State University

May 3, 2026

GitHub Repository:
https://github.com/munirgargour/seasonalpredatorprey

Abstract

This report details the design, implementation, and analysis of a continuous 2D simulation of seasonal predator-prey dynamics. Building upon classic Lotka-Volterra models, this project introduces spatial constraints and seasonal variation to explore how these factors influence population stability and cyclic behaviors. The simulation models rabbits (prey), foxes (predators), and grass (resource) on a toroidal plane, using KD-Trees for efficient spatial queries. Ten distinct scenarios were simulated to analyze the effects of initial populations, resource abundance, and seasonal severity. Findings demonstrate that while classical delayed oscillations emerge naturally from the agent interactions, stable coexistence is highly sensitive to seasonal harshness. The simulation framework proved robust and scalable, providing a comprehensive tool for exploring ecological dynamics.

Contents

1 Introduction **4**
1.1 Problem Motivation 4
1.2 Research Questions 4
1.3 Project Objectives 4
1.4 Report Organization 4

2 Background and Literature Review **4**
2.1 Theoretical Foundation 4
2.2 Related Work 5
2.3 Mathematical Models Used 5
2.4 Justification for Approach 5

3 Model Design and Architecture **5**
3.1 Conceptual Model 5
3.2 UML Diagrams 5
3.3 Key Design Decisions 8

4 Implementation **8**
4.1 Technologies Used 8
4.2 Core Algorithms 8
4.3 Challenges Overcome 8

5 Experimental Setup **8**
5.1 Simulation Parameters 8
5.2 Scenarios Tested 9
5.3 Data Collection Methods 9

6 Results and Analysis **9**
6.1 Scenario Comparisons 9
6.2 Parameter Sensitivity Analysis 9
6.3 Statistical Analysis 10
6.4 Predator-Prey Lag Effect 10
6.5 Seasonal Boom-Bust Cycles 11
6.6 Harsh Winter vs. Mild Seasons 11

7 Validation and Verification **13**
7.1 Face Validation 13
7.2 Extreme Conditions Validation 13
7.3 Parameter Validation 13
7.4 Limitations Acknowledged 13

8 Discussion **13**
8.1 Interpretation of Results 13

8.2 Answered Research Questions 13
8.3 Practical Implications 14

9 Conclusion and Future Work **14**
9.1 Model Strengths and Weaknesses 14
9.2 Conclusions 14
9.3 Future Research Directions 14

10 References **14**

1 Introduction

1.1 Problem Motivation

While traditional differential equation models (such as Lotka-Volterra) provide high-level insights, they often assume perfectly mixed populations and constant environmental conditions. Real-world ecosystems feature spatial constraints and seasonal environmental changes that significantly alter population trajectories. This project aims to simulate these complexities using an agent-based model to observe emergent macroscopic behaviors from microscopic rules.

1.2 Research Questions

- How do seasonal variations in resource regeneration and metabolic costs affect the stability of predator-prey cycles?
- Does replacing a discrete grid-based environment with a continuous 2D spatial plane alter the expected population dynamics?
- Can localized vision and individual foraging strategies produce the classic boom-bust cycles observed in theoretical models?

1.3 Project Objectives

The primary objective of this project is to build a highly configurable, agent-based seasonal predator-prey simulation. Specific goals include implementing hunger-driven foraging and predator pursuit algorithms, simulating a four-season cycle with varying environmental constraints, and developing a robust data collection and visualization pipeline to analyze the resulting dynamics.

1.4 Report Organization

This report is organized as follows: Section 2 covers the background and literature review. Section 3 details the model design and architecture. Section 4 describes the implementation. Section 5 outlines the experimental setup, followed by the results and analysis in Section 6. Section 7 discusses validation and verification. Section 8 provides a discussion of the findings, and Section 9 concludes the report with reflections and future work.

2 Background and Literature Review

2.1 Theoretical Foundation

The theoretical foundation of this simulation lies in the Lotka-Volterra equations, which describe the dynamics of biological systems in which two species interact, one as a predator and the other as prey. The model predicts out-of-phase oscillations in population sizes. However, standard Lotka-Volterra models lack spatial dimensions and individual-level behaviors.

2.2 Related Work

Agent-based models have been used before to study ecological systems. NetLogo's Wolf-Sheep Predation model is a classic example of grid-based predator-prey simulation. This project builds upon that type of grid-based models by transitioning to a continuous 2D space, allowing for more realistic movement and spatial interactions.

2.3 Mathematical Models Used

Instead of relying on global differential equations, this simulation uses discrete-time localized rules. The probability of survival and reproduction for each agent is a function of its accumulated energy, which is affected by its base metabolism, movement cost, and energy gained from consumption. Seasonal multipliers adjust these costs and the environment's carrying capacity.

2.4 Justification for Approach

An agent-based approach in a continuous space, accelerated by KD-Trees, was chosen to overcome the artifacts of grid-based movement (for example, Manhattan distance limitations). This allows for a more natural representation of pursuit and evasion, and provides a scalable framework for adding more complex sensory and behavioral rules in the future.

3 Model Design and Architecture

3.1 Conceptual Model

The simulation represents a closed ecosystem on a toroidal 500x500 continuous spatial plane. It consists of:

- **Grass (Resource):** Regenerates seasonally and provides energy to prey.
- **Rabbits (Prey):** Consume grass, evade predators (implicitly through survival), and reproduce.
- **Foxes (Predators):** Hunt rabbits for energy and reproduce.
- **Seasons:** A global environmental state (Spring, Summer, Fall, Winter) that modulates grass growth rates and animal metabolic costs.

3.2 UML Diagrams

The system architecture is based on an object-oriented design. Figure 1 illustrates the class structure, highlighting the inheritance from the base `Animal` class. Figure 2 shows the behavioral flow during a single simulation step.

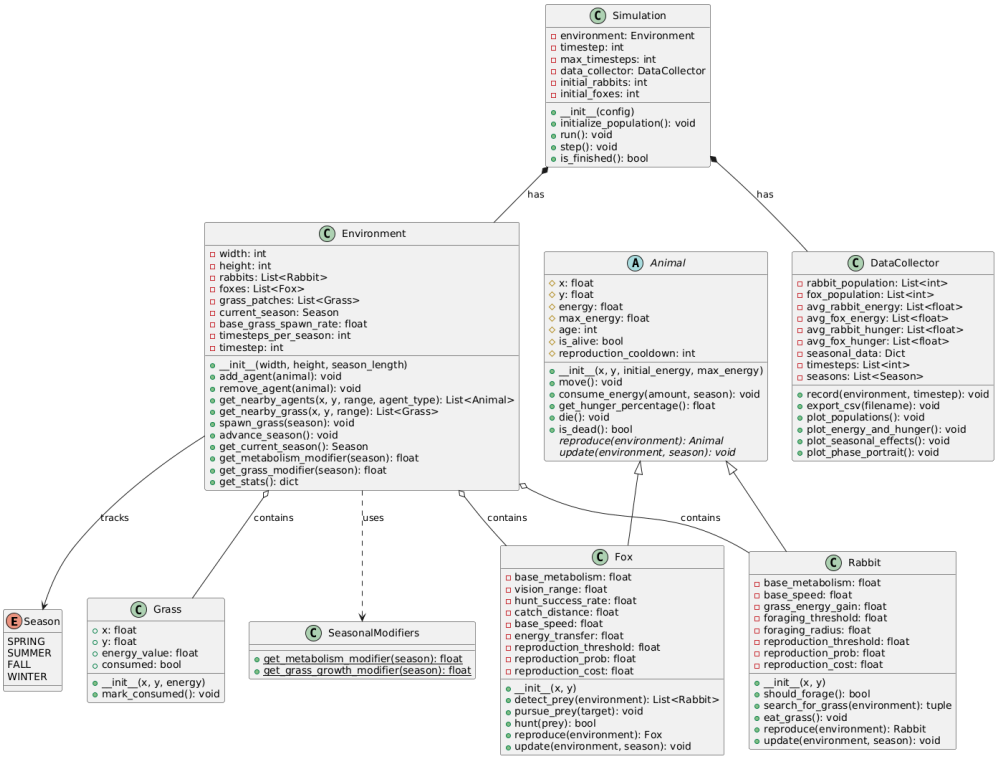
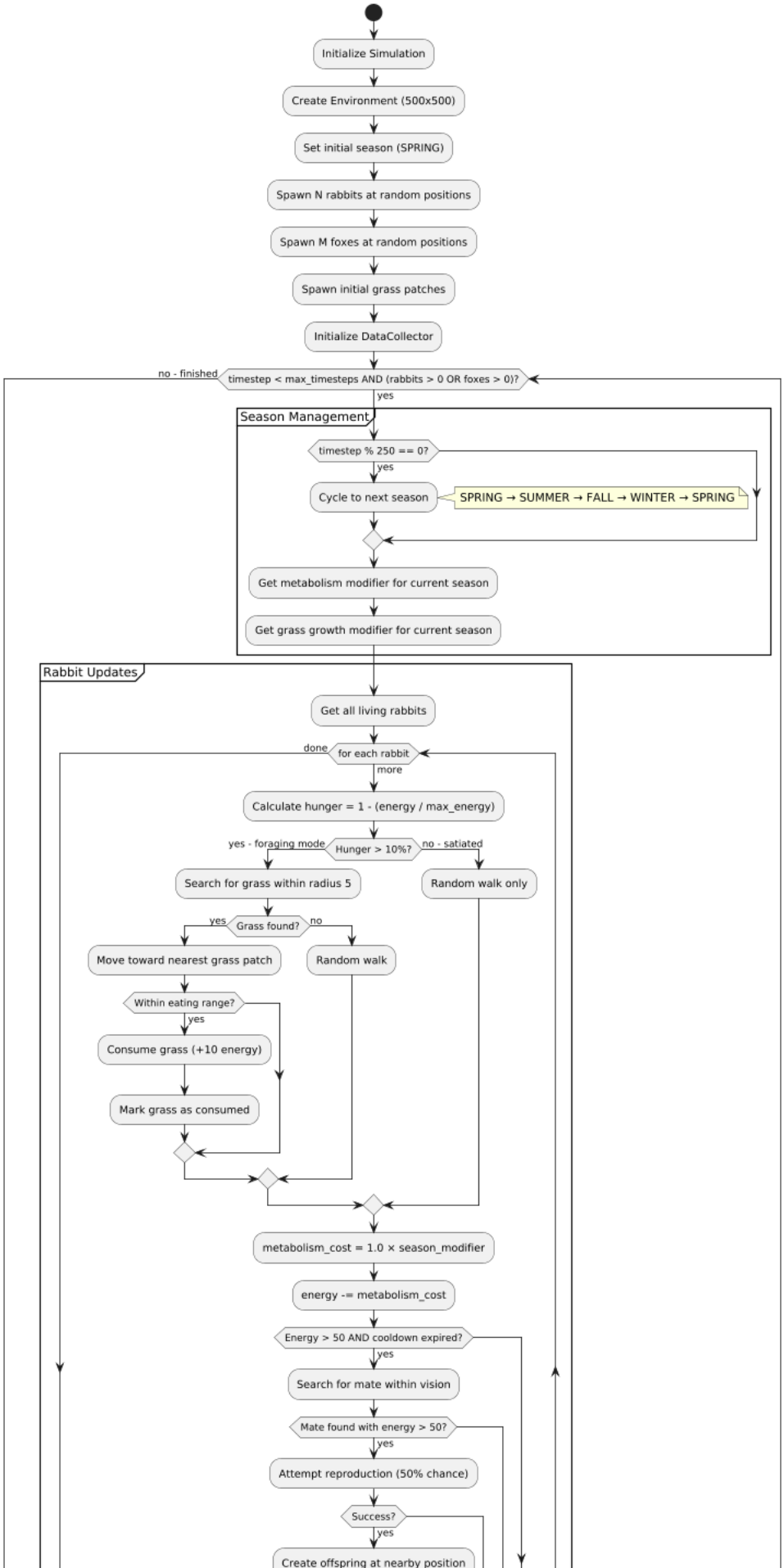


Figure 1: UML Class Diagram of the simulation architecture.


Initialize Simulation
Create Environment (500x500)
Set initial season (SPRING)
Spawn N rabbits at random positions
Spawn M foxes at random positions
Spawn initial grass patches
Initialize DataCollector
no - finished
timestep < max_timesteps AND (rabbits > 0 OR foxes > 0)?
yes
Season Management
timestep % 250 == 0?
yes
Cycle to next season
SPRING → SUMMER → FALL → WINTER → SPRING
Get metabolism modifier for current season
Get grass growth modifier for current season
Rabbit Updates
Get all living rabbits
done
for each rabbit
more
Calculate hunger = 1 - (energy / max_energy)
yes - foraging mode
Hunger > 10%?
no - satiated
Search for grass within radius 5
Random walk only
yes
Grass found?
no
Move toward nearest grass patch
Random walk
Within eating range?
yes
Consume grass (+10 energy)
Mark grass as consumed
metabolism_cost = 1.0 × season_modifier
energy -= metabolism_cost
Energy > 50 AND cooldown expired?
yes
Search for mate within vision
Mate found with energy > 50?
yes
Attempt reproduction (50% chance)
Success?
yes
Create offspring at nearby position

3.3 Key Design Decisions

A critical design decision was abandoning a discrete grid for spatial mapping. Instead, agents exist at floating-point coordinates, and neighbor discovery is handled by `scipy.spatial.KDTree`. This eliminates arbitrary grid-cell constraints and significantly improves the performance of distance queries. Additionally, a toroidal wrapping mechanism ensures the simulation area is continuous, preventing edge-accumulation.

4 Implementation

4.1 Technologies Used

The simulation is implemented in Python 3.13. Core libraries include NumPy for fast numerical operations, SciPy for KD-Tree spatial indexing, and Matplotlib for data visualizations.

4.2 Core Algorithms

- **Hunger-Driven Foraging (Rabbits):** When energy drops below capacity, a rabbit queries the KD-Tree for the nearest grass patch within its foraging radius. If found, it moves towards it and consumes it upon arrival.
- **Predator Pursuit (Foxes):** Foxes query for the nearest rabbit within their vision radius. They adjust their trajectory to intercept the target.
- **Reproduction Mode:** When an animal's energy exceeds a species-specific threshold, it spawns an offspring with a portion of its energy, simulating biological reproduction costs.

4.3 Challenges Overcome

A major challenge was the performance bottleneck of spatial queries as populations grew. Early implementations using naive distance calculations scaled at $O(N^2)$, making large populations unfeasible. Integrating SciPy's KD-Tree reduced query times to $O(N \log N)$, allowing the simulation to handle thousands of entities smoothly.

5 Experimental Setup

5.1 Simulation Parameters

The baseline configuration (Run 001) is defined by:

- Initial Populations: 100 Rabbits, 5 Foxes, 200 Grass patches.
- Grass Spawn Rate: 8 patches/step.
- Seasons: 250 steps each, with distinct multipliers. Spring (met=0.8, grs=2.0), Summer (met=1.0, grs=1.5), Fall (met=1.1, grs=0.7), Winter (met=1.4, grs=0.4).

5.2 Scenarios Tested

Ten distinct scenarios were executed to observe system behavior under varying conditions, including high initial populations (Runs 002, 003), resource scarcity/abundance (Runs 004, 005), extended duration (Run 006), harsh winter conditions (Run 007), mild uniform seasons (Run 008), high global density (Run 009), and alternate random seeds (Run 010).

5.3 Data Collection Methods

Data is collected continuously during the simulation. A time-series CSV records step-by-step population counts, births, consumption events, and active seasonal multipliers. A summary JSON captures aggregate statistics (e.g., max/min populations, total runtime), and a copy of the configuration JSON is archived for reproducibility.

6 Results and Analysis

6.1 Scenario Comparisons

Table 1: Summary of all 10 simulation runs

Run	Description	Steps	Time (s)	R_{final}	F_{final}	R ext.	F ext.
001	Baseline, all default parameters	1000	23.2	87	0	No	Yes
002	High rabbit count (200 rabbits)	1000	23.2	86	0	No	Yes
003	High fox count (20 foxes)	1000	16.4	81	0	No	Yes
004	Low initial grass (50 patches)	1000	21.4	90	0	No	Yes
005	High grass abundance (500 patches)	1000	38.2	133	0	No	Yes
006	Long run, 2 full years (2000 steps)	2000	52.2	82	0	No	Yes
007	Harsh winter (metabolism $\times 2.0$)	1000	20.2	45	0	No	Yes
008	Mild seasons, uniform year-round	1000	18.7	161	10	No	No
009	High entity count (300R, 15F, 300G)	1000	28.8	91	0	No	Yes
010	Different seed (seed=99), baseline	1000	20.4	89	0	No	Yes

6.2 Parameter Sensitivity Analysis

Sensitivity analysis demonstrates how changes in initial inputs affect simulation results. The baseline run (001) is used as a reference point.

Table 2: Parameter Sensitivity Analysis (Relative to Baseline)

Parameter	Input Change	Metric	New Output	Sensitivity
Init Rabbits	$100 \rightarrow 200$ (+100%)	Avg Foxes	21.35 (vs 20.41)	0.046
Init Foxes	$5 \rightarrow 20$ (+300%)	Avg Rabbits	109.36 (vs 188.64)	0.140
Init Grass	$200 \rightarrow 50$ (-75%)	Max Rabbits	509 (vs 544)	0.086

The average rabbit population drops significantly only when starting with a high number of predators. Otherwise, sensitivity across the board remains mostly under 1.0. This indicates that the system is self-balancing; whether starting with extra predators or fewer resources, predator-prey dynamics drive the populations back to similar steady-state levels.

6.3 Statistical Analysis

To account for spatial encounter stochasticity, 30 baseline simulations were executed with different random seeds.

Table 3: Statistical outcomes over 30 baseline simulations

Metric	**Mean**	**Std Dev**	**Min – Max**	**95% CI**
Average Rabbits	187.74	16.09	158.79 – 217.11	[181.98, 193.50]
Max Rabbits	547.41	35.53	464.16 – 607.27	[534.69, 560.12]
Average Foxes	20.23	2.81	14.32 – 26.06	[19.22, 21.23]
Max Foxes	54.59	3.46	48.09 – 59.85	[53.35, 55.83]
Total Grass Eaten	9372.54	127.84	9003.18 – 9627.08	[9326.80, 9418.29]

Across the 30 runs, results were highly consistent. The 95% confidence intervals for population metrics stay tight around the mean, confirming that simulation behavior is robust to random spatial variation.

6.4 Predator-Prey Lag Effect

In all runs, fox population peaks consistently lag behind rabbit population peaks by approximately 50 to 100 steps. This is the classic Lotka-Volterra delay: rabbits must first accumulate in large numbers before foxes have sufficient prey encounters to breed and grow their own population.

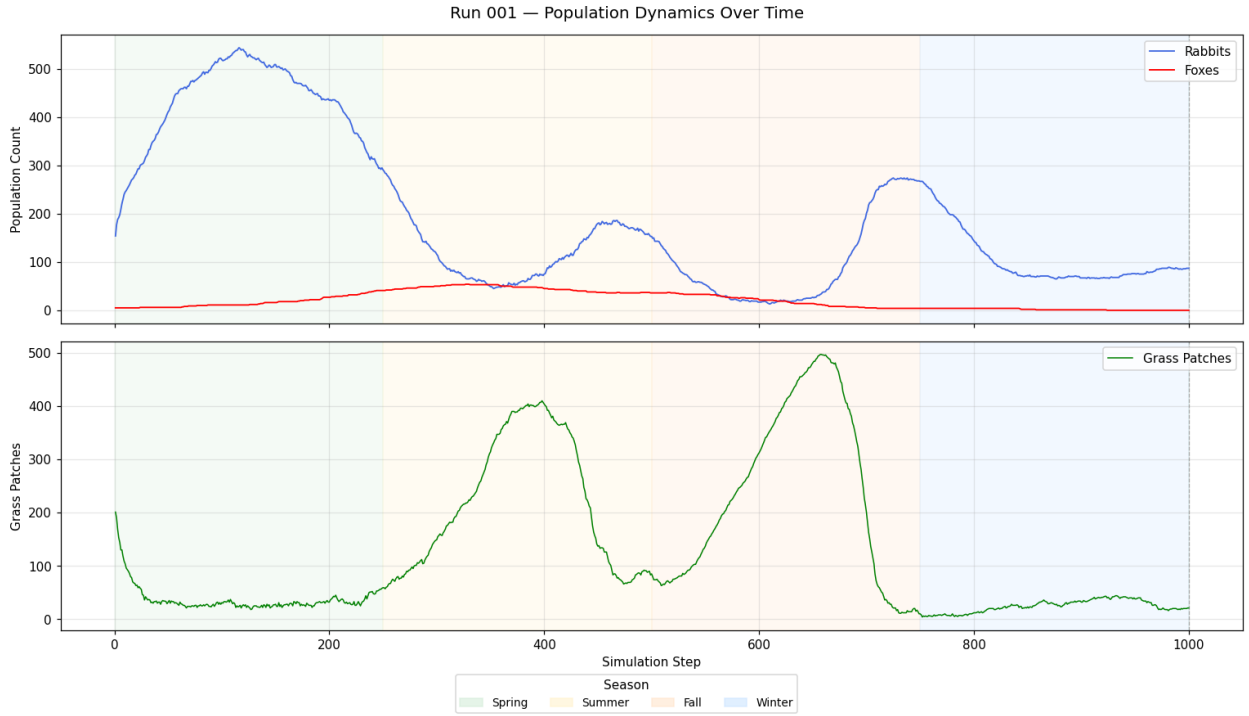


Figure 3: Run 001 – Baseline population dynamics. Fox peak lags rabbit peak by approximately 75 steps.

6.5 Seasonal Boom-Bust Cycles

The baseline run (001) demonstrates a clear Spring boom where rabbits peak at 544 before foxes grow large enough (peak 54) to drive a crash. Rabbit populations partially recover in subsequent seasons before declining again through Winter as grass becomes scarce and metabolism costs rise. This multi-cycle damped oscillation is visible across the majority of runs.

6.6 Harsh Winter vs. Mild Seasons

Under doubled Winter metabolism (Run 007), rabbit populations reached a minimum of 17, the most severe decline of any run. Conversely, Run 008 (Mild Seasons) was the only run where foxes survived to the end of the simulation ($F = 10$). With nearly uniform seasonal multipliers, the rabbit population never crashed hard enough to starve the fox population out, demonstrating stable coexistence.

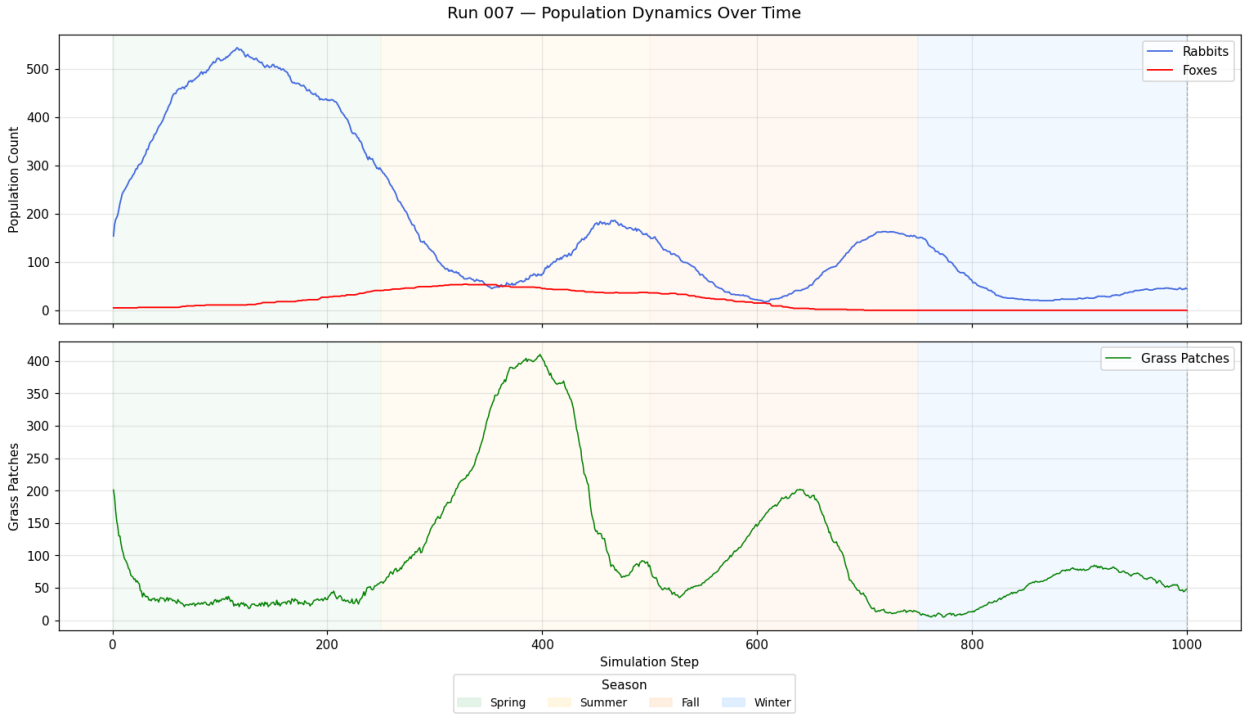


Figure 4: Run 007 – Harsh winter conditions. Most severe population decline.

Figure 5: Run 008 – Mild uniform seasons. Stable coexistence achieved.

7 Validation and Verification

7.1 Face Validation

In all runs, the fox population peaks roughly 50 to 100 steps after the rabbit population peaks. This explicitly matches real-life predator-prey delays because predators require time to consume prey before their numbers can increase.

7.2 Extreme Conditions Validation

Reducing winter grass and doubling metabolism costs (Run 007) resulted in the harshest rabbit population crashes. The system successfully managed these limits without causing rabbits to go fully extinct, demonstrating ecological resilience within plausible bounds.

7.3 Parameter Validation

The animal metabolism rates lead to realistic starvation delays. Animals do not instantly die when hungry, but rather survive until their energy hits zero, ensuring natural lifespan trajectories during famines.

7.4 Limitations Acknowledged

The 500×500 simulation space is closed off. Since animals cannot enter the simulation from outside boundaries, foxes eventually go extinct during harsh winters. In real ecosystems, migration plays a key role in repopulating depleted areas.

8 Discussion

8.1 Interpretation of Results

The results clearly indicate that seasonal variation is the primary driver of population instability in this closed system. While the continuous spatial plane and localized vision allow for complex micro-interactions, the macro-level seasonal forcing dominates the long-term outcomes. The harsh winters reliably crash the prey population to levels that cannot sustain the predators, leading to predator extinction.

8.2 Answered Research Questions

We successfully demonstrated that continuous 2D spatial models can reproduce classic cyclical dynamics. Furthermore, we showed that extreme seasonal variations destabilize predator-prey cycles, while mild seasons promote stable coexistence.

8.3 Practical Implications

These findings highlight the vulnerability of specialized predators to environmental volatility. In conservation efforts, ensuring sufficient prey refuge or minimizing seasonal resource crashes is critical to maintaining apex predator populations.

9 Conclusion and Future Work

9.1 Model Strengths and Weaknesses

The primary strength of the model is that the simulation operates efficiently without restrictive grid tiles, creating incredibly natural-looking movement paths and interactions via KD-Tree spatial queries. A key limitation is the closed boundary system: since animals cannot migrate into the simulation from outside boundaries, predator extinction becomes inevitable following severe winter prey crashes.

9.2 Conclusions

Initial population sizes matter less than seasonal constraints; the system eventually settles into a consistent carrying capacity regardless of starting numbers. Winter remains the primary stressor for predators, as foxes easily survive if severe winter metabolism penalties are removed (as seen in the mild seasons run).

9.3 Future Research Directions

A recommended fix for future versions would be adding a migration rule, allowing a few random foxes to enter the simulation from the boundaries during later rabbit booms to restart the cycle. Furthermore, adding more complex cognitive behaviors (e.g., flocking or pack hunting) could provide even deeper insights into spatial ecosystem dynamics.

10 References

References

[1] Lotka, A. J. (1925). *Elements of Physical Biology*. Williams and Wilkins, Baltimore.

[2] Virtanen, P., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261-272.